

The pretraining data consists of random bigram samples (X_0, X_1) where $X_1 \sim p^\varepsilon(\cdot|X_0)$; we allow X_0 to be either uniform over S or distributed according to the stationary measure π^ε of p^ε . The latter arises when generating samples $(X_{t-1}, X_t)_{t \geq 1}$ from the observed transitions of the (unbounded) chain $(X_t^\varepsilon)_{t \geq 0}$. The model is trained by gradient descent with cross-entropy loss, with an intermediate thresholding step to mask out edges determined to not lie in E . See Algorithm 1 for details and Theorem D.1 for the full statement.

Theorem 3.1 (convergence of pretraining). *Let $X_0 \sim \text{Unif}(S)$ or $X_0 \sim \pi^\varepsilon$ and $X_1 \sim p^\varepsilon(\cdot|X_0)$ be random samples from X^ε . Then for the gradient descent iterates $\mathbf{W}^{(t)}$ from Algorithm 1 w.r.t. cross-entropy loss*

$$L_{\text{pre}}(\mathbf{W}) = \mathbb{E}_{X_0, X_1}[-\log \hat{p}_{\mathbf{W}}(X_1|X_0)],$$

the learned transition probabilities $\hat{p}_{ij}^{(T)} = \hat{p}_{\mathbf{W}^{(T)}}(e_j|e_i)$ converge with error $\sup_{i,j} |\hat{p}_{ij}^{(T)} - p_{ij}^\varepsilon| = O(\sqrt{KM^2/T})$ before thresholding. Moreover, after thresholding at time $T_1 = \tilde{O}(KM^2\varepsilon^{-2})$, the error converges as $\exp(-\Omega(\varepsilon^2 T))$. Hence after $T_2 = \tilde{O}(KM\varepsilon^{-2})$ additional steps, the output of Algorithm 1 has error $\exp(-\Omega(|S|))$.

Thus the base model $\hat{p}$ learns the underlying graph E and all transition probabilities with exponentially small error. Under mild regularity conditions, all assumptions can be verified for $\hat{p}$ (see Propositions B.8 and C.8); to simplify the discussion, we henceforth assume the base model is exact, $\hat{p} = p^\varepsilon$. We remark that while the time to converge is quite long compared to the search, RL and distillation methods studied later, this is natural as pretraining is done on much longer timescales compared to test-time compute.

3.2 Learning Sparse Rewards via Search

Having learned the underlying probabilities p^ε , the base model now performs CoT reasoning by generating each step of the chain $(X_t^\varepsilon)_{t \geq 0}$ in sequence starting from $X_0^\varepsilon = X_{\text{in}}$. Since the reasoner has no prior knowledge of which steps it must take to progress towards X_{out} , on average it will spend a long time trapped in each cluster before chancing upon a sparse edge (new idea) and moving to a new cluster. From our quantitative dynamical analysis, we are able to obtain a nearly tight characterization of the average hitting time.

Theorem 3.2 (expected hitting time). *Under Assumptions 1-4, it holds for all $\varepsilon \leq \varepsilon_{\max} := \Theta(M^{-1}(\log M)^{-4})$ that*

$$\mathbb{E}_{(X_{\text{in}}, X_{\text{out}}) \sim \mathcal{D}}[\mathbb{E}_{X_{\text{in}}}[\tau_{X_{\text{out}}}^\varepsilon]] = \tilde{\Theta}\left(\frac{KM}{\varepsilon}\right).$$

Intuitively, since each cluster is rapidly mixing, the chain will spend roughly $\Theta(1/M)$ of the time in states with outbound edges, from where it escapes with probability $\Theta(\varepsilon)$. Such rare events are distributed approximately exponentially, and must be repeated $\Theta(K)$ times to reach the cluster containing X_{out} , where the chain will mix fast and likely hit X_{out} .

This result also illustrates a simple method to improve the hitting time: modifying the underlying probabilities to increase the denominator ε . This corresponds to guiding CoT or fine-tuning the base model so that (correct) new, difficult reasoning steps are generated more often, ensuring a more efficient exploration of the solution space. However, this cannot be done by simply increasing the likelihood of low-probability edges, as there may be many low-probability edges within clusters as well; we want to only boost the generation of sparse edges to preserve the capabilities of the pretrained model. Indeed, we demonstrate in Theorem 5.3 that any updates based on local information is not enough to improve reasoning ability in a precise sense.

Instead, we run a simple *tree search* protocol to identify sparse edges, detailed in Algorithm 2. The method consists of randomly sampling a state X_0 and rolling out N random walks in parallel to construct an estimate $\hat{C}$ of the cluster containing X_0 for time T_0 . After the cluster has been sufficiently explored we continue to simulate each walk until a transition outside $\hat{C}$ is detected, at which point the edge is marked as a sparse edge (added to $\hat{E}$) and the path is terminated. This continues until all paths are terminated or a time horizon $T_{\max}$ is reached. Since we are not receiving signals from an external oracle but rather recording rare transitions, this is similar to intrinsic rewards such as curiosity or exploration bonuses (Burda et al., 2018, 2019).

We consider two versions of this process, **PRM mode** and **RL mode**, depending on whether the information gained from search is collected into an external reward model or used to fine-tune the base model. The benefits of both methods for reasoning is discussed in the next subsection.

3.3 Improving the Base Model via RL

PRM mode keeps an external process reward ‘model’ (PRM) throughout the search process, which is simply the set $\mathcal{M}_s$ which collects the estimated sparse edges over multiple iterations of the outer loop to reconstruct E_s . We prove that the PRM is strongly consistent:

Proposition 3.3. *PRM mode of Algorithm 2 returns $\mathcal{M}_s = E_s$ with probability $1 - \tilde{O}(1/K)$.*

Then by increasing the likelihood of transitions $(x, y) \in \mathcal{M}_s$ when the current state is x by a factor of $\varepsilon_{\max}/\varepsilon$, the PRM can guide CoT to follow $p^{\varepsilon_{\max}}$ rather than p^ε . It is immediate from Theorem 3.2 that the expected hitting time decreases from $\tilde{\Theta}(KM/\varepsilon)$ to $\tilde{\Theta}(KM/\varepsilon_{\max})$. Moreover, the time complexity of Algorithm 2 is $RT_{\max} = \tilde{O}(KM/\varepsilon)$, which is equal to the time to solve a *single* instance $(X_{\text{in}}, X_{\text{out}})$ *without* search, and the memory requirement is only $O(M + K)$. This demonstrates the effectiveness of utilizing search to guide CoT generation.

However, it is often desirable to use the information gained during search to directly fine-tune the pretrained model, so that maintaining an independent PRM is not necessary. **RL mode** performs online RL updates to $\hat{p}_{\mathbf{W}}$ at each iteration of Algorithm 2; while many policy gradient methods can be applied, we analyze the popular proximal policy optimization algorithm (PPO-Clip, Schulman et al., 2017). Based on the estimate $\hat{E}$ of

Algorithm 2 Sparse Edge Search

Require: pretrained model $\hat{p}_{\mathbf{w}}$

- 1: set $R = \Theta(K \log K)$, $N = \Theta(\log K)$,
 $T_0 = \Theta(M(\log M)^2)$, $T_{\max} = \Theta(M/\varepsilon)$, $\mathcal{M}_s = \emptyset$
- 2: **for** $r = 1, \dots, R$ **do**
- 3: set $\hat{C}, \hat{C}^n, \hat{E} = \emptyset$, $A = [N]$
- 4: sample $X_0 \sim \text{Unif}(S)$ or $X_0 \sim \pi^\varepsilon$
- 5: **for** $t = 1, \dots, T_{\max}$ **do**
- 6: **for** $n \in A$ **do**
- 7: generate $X_t^{n,\varepsilon} \sim \hat{p}_{\mathbf{w}}(\cdot | X_{t-1}^{n,\varepsilon})$
- 8: **if** $t \leq T_0$ **then** (cluster search)
- 9: $\hat{C}^n \leftarrow \hat{C}^n \cup \{X_t^{n,\varepsilon}\}$
- 10: **else if** $t > T_0$, $X_t^{n,\varepsilon} \notin \hat{C}^n$ **then** (edge search)
- 11: $\hat{E} \leftarrow \hat{E} \cup \{(X_{t-1}^{n,\varepsilon}, X_t^{n,\varepsilon})\}$
- 12: $A \leftarrow A \setminus \{n\}$
- 13: **end if**
- 14: **end for**
- 15: **if** $t = T_0$ **then**
- 16: $\hat{C} = \cap_{n=1}^N \hat{C}^n$
- 17: **end if**
- 18: **end for**
- 19: run Algorithm 3 with $\hat{p}_{\mathbf{w}}, \hat{E}$ (if RL mode)
- 20: $\mathcal{M}_s \leftarrow \mathcal{M}_s \cup \hat{E}$ (if PRM mode)
- 21: **end for**
- 22: return $\mathcal{M}_s$

sparse edges originating from the initialized cluster, we define the advantage function as $\hat{A}(x, y) = 1$ if $(x, y) \in \hat{E}$ and 0 otherwise. Similarly to pretraining, samples are generated as $X_0 \sim \text{Unif}(S)$ or π^ε and $X_1 \sim p^\varepsilon(\cdot | X_0)$. The objective of PPO-Clip to be maximized is (OpenAI, 2018)

$$L_{\text{PPO}}(\mathbf{W}; \hat{A}) = \mathbb{E}_{X_0, X_1} \left[\min \left\{ \frac{\hat{p}_{\mathbf{w}}(X_1 | X_0)}{p^\varepsilon(X_1 | X_0)}, c_{\text{clip}} \right\} \hat{A}(X_0, X_1) \right].$$

The old policy is fixed to p^ε during Algorithm 2. We use sign gradient descent for simplicity of analysis (ordinary gradient descent also guarantees convergence as long as $\varepsilon \geq \varepsilon_{\max}^2$).

Proposition 3.4 (convergence of PPO-Clip). *By running RL mode of Algorithm 2 with PPO-Clip, the base model p^ε is modified to $p^{\varepsilon'}$ where $\varepsilon' = (1 - o(1))\varepsilon_{\max}$ with probability $1 - \tilde{O}(1/K)$.*

The additional time complexity of running PPO-Clip is $\tilde{O}(K \log(\varepsilon_{\max}/\varepsilon))$ which is small compared to the pretraining time or search process. In particular, it again follows